VIKNESH.G 23MID0161

Advance Python Digital Assignment

Student class (academic performance tracker) Gene class (DNA sequence handler)

Problem1: Student class

In [3]:
```python
class Student:


    def __init__(self, name, student_id):

        self.name = name
        self.student_id = student_id
        self.scores = []
# ADD SCORE METHOD
    def add_score(self, score):
        self.scores.append(score)
        print(f"Added score {score} for {self.name}. Current scores: {self.scores}")
#AVG CALC.
    def calculate_average(self):
        if len(self.scores) == 0:
            return 0
        total = sum(self.scores)
        avg = total / len(self.scores)
        return round(avg, 2)
#GRADE CALC.
    def get_final_grade(self):
        avg = self.calculate_average()
        if avg >= 60:
            return "Pass"
        else:
            return "Fail"
```

Test Case 1 - Creating a student object

In [4]:
```
student1 = Student("Alice", 101)
print(student1.name)
print(student1.student_id)
print(student1.scores)
```

```
Alice
101
[]
```

Test Case 2 - Adding a few scores

In [5]:
```
#ADDINF SCORE TO ALICE


student1.add_score(75)
student1.add_score(85)
student1.add_score(55)
print(student1.scores)
```

```
Added score 75 for Alice. Current scores: [75]
Added score 85 for Alice. Current scores: [75, 85]
Added score 55 for Alice. Current scores: [75, 85, 55]
[75, 85, 55]
```

Test Case 3 - Average

In [6]:
```
print(student1.calculate_average())
```

```
71.67
```

Test Case 4 - Final grade (Pass case)

In [7]:
```
#GARDE
print(student1.get_final_grade())
```

```
Pass
```

Test Case 5 - Final grade (Fail case)

In [8]:
```
student2 = Student("Bob", 102)
student2.add_score(40)
student2.add_score(50)
print(student2.calculate_average())
print(student2.get_final_grade())
```

```
Added score 40 for Bob. Current scores: [40]
Added score 50 for Bob. Current scores: [40, 50]
45.0
Fail
```

Test Case 6 - No scores at all

In [10]:
```
# SCORE IS NOT INITIALIZED SO ITS EMPTY AND CALCULATED RESULT
student3 = Student("Charlie", 103)
print(student3.calculate_average())
print(student3.get_final_grade())
```

```
0
Fail
```

In [11]:
```
student4 = Student("Diana", 104)
student4.add_score(90)
student4.add_score(70)
# whoops - typing the wrong attribute name
print(student4.score)
```

```
Added score 90 for Diana. Current scores: [90]
Added score 70 for Diana. Current scores: [90, 70]
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[11], line 5
      3 student4.add_score(70)
      4 # whoops - typing the wrong attribute name
----> 5 print(student4.score)

AttributeError: 'Student' object has no attribute 'score'
```

Problem2: Gene

In [12]:
```
class Gene:


    def __init__(self, gene_id, sequence):
        print("I'm a new Gene object!")
        print(f"My constructor got a param: {gene_id}")
        print("Assign that param to my id instance variable...")
        self.id = gene_id
        print("Similarly, assigning to my sequence instance variable...")

        self.set_seq(sequence)

    def print_id(self):
        print(f"My id is: {self.id}")

    def print_sequence(self):
        print(f"My sequence is: {self.sequence}")

    def base_composition(self, base):

        return self.sequence.count(base)

    def gc_content(self):
        g_count = self.base_composition("G")
        c_count = self.base_composition("C")
        gc_total = g_count + c_count
        return gc_total / len(self.sequence)

    def get_seq(self):
        return self.sequence

    def set_seq(self, new_sequence):

        assert "U" not in new_sequence.upper(), "Sorry, no RNA allowed."
        self.sequence = new_sequence
```

TC 1 - Creating a gene object

In [13]:
```
gene1 = Gene(101, "ATGCGT")
```

```
I'm a new Gene object!
My constructor got a param: 101
Assign that param to my id instance variable...
Similarly, assigning to my sequence instance variable...
```

TC2 - Printing id and sequence

In [14]:
```
gene1.print_id()
gene1.print_sequence()
```

```
My id is: 101
My sequence is: ATGCGT
```

TC 3 - Base composition

In [15]:
```
print(gene1.base_composition("A"))
print(gene1.base_composition("G"))
print(gene1.base_composition("T"))
```

```
1
2
2
```

TC 4 - GC content

In [16]:
```
print(gene1.gc_content())
```

```
0.5
```

TC 5 - Get sequence

In [19]:
```
print(gene1.get_seq())
# RETURNS THE DNA SEQUENCE
```

```
ATGCGT
```

TC 6 - Set sequence (valid DNA)

In [20]:
```
gene1.set_seq("ATATGC")
print(gene1.get_seq())
```

```
ATATGC
```

Test Case 7 - Set sequence (invalid RNA)

This one is *supposed* to raise an AssertionError - that's the correct behaviour, not a bug.

In [21]:
```
gene1.set_seq("AUGC")
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[21], line 1
----> 1 gene1.set_seq(      )

Cell In[12], line 34, in Gene.set_seq(self, new_sequence)
     32 def set_seq(self, new_sequence):
     33     # DNA only - no U (uracil) allowed, that's an RNA base
---> 34     assert "U" not in new_sequence.upper(), "Sorry, no RNA allowed."
     35     self.sequence = new_sequence

AssertionError: Sorry, no RNA allowed.
```

In [25]:
```
print(gene1.set_seq("ATGC"))
```

```
None
```

In [ ]:

In [ ]:

VIKNESH.G ||| 23MID0161 ||| 24TH AUGEST 2026

TASK 1 CREATION OF THREAD

In [1]:
```
import time
import threading
start = time.perf_counter()

def do_something():
    print ('sleeping for 2 seconds')
    time.sleep(2)
    print('Done sleeping...')

t1 = threading.Thread(target = do_something)
t1.start()
t1.join()
t2 = threading.Thread(target = do_something)
t2.start()
t2.join()
stop= time.perf_counter()
print("total time taken to run 2 thrteads : {} ".format(stop-start))
```

```
sleeping for 2 seconds
Done sleeping...
sleeping for 2 seconds
Done sleeping...
total time taken to run 2 thrteads : 4.023728200001642
```

process of creating thread one by one sequentially which shows each thread sleeps of 2 second hence there is 2 threds running which consumes 4 seconds to complete both the task

TASK 2 CREATION OF MULTIPLE THREADS STORING IT IN A LIST

In [2]:
```
start = time.perf_counter()
threads = []
for _ in range(50):
    t1 = threading.Thread(target = do_something)
    t1.start()
    threads.append(t1)

for i in threads:
    i.join()

stop = time.perf_counter()

print("total time taken to run 2 thrteads : {} ".format(stop-start))
```

```
sleeping for 2 seconds
sleeping for 2 seconds
sleeping for 2 seconds
```

```
sleeping for 2 seconds
sleeping for 2 seconds
sleeping for 2 seconds
sleeping for 2 seconds
sleeping for 2 seconds
sleeping for 2 seconds
sleeping for 2 seconds
sleeping for 2 seconds
Done sleeping...
Done sleeping...
Done sleeping...
```

```
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...

total time taken to run 2 thrteads : 2.246801699977368
```

TASK 3 ||| PASSING ARGS

In [3]:
```python
start = time.perf_counter()

def do_something(seconds):
    print (f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    print('Done sleeping...')

t1 = threading.Thread(target = do_something) ### USE OF ARGS

t1.start()
t1.join()
t1 = threading.Thread(target = do_something, args = [5])

t1.start()
t1.join()
stop = time.perf_counter()

print("total time taken to run 2 thrteads : {} ".format(stop-start))
```

```
Exception in thread Thread-55 (do_something):
Traceback (most recent call last):
  File "C:\Users\Viknesh G\anaconda3\Lib\threading.py", line 1043, in _bootstrap_inner
    self.run()
    ~~~~~~~~^^
  File "C:\Users\Viknesh G\anaconda3\Lib\site-packages\ipykernel\ipkernel.py", line 772, in run_closure
    _threading_Thread_run(self)
    ~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\Viknesh G\anaconda3\Lib\threading.py", line 994, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: do_something() missing 1 required positional argument: 'seconds'
sleeping 5 second (s)
Done sleeping...
total time taken to run 2 thrteads : 5.065460600017104
```

In [4]:

```
start = time.perf_counter()

def do_something(seconds):
    print (f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    print('Done sleeping...')

t1 = threading.Thread(target = do_something, args = [5])

t1.start()
t1.join()
stop = time.perf_counter()

print("total time taken to run 2 thrteads : {} ".format(stop-start))
```

```
sleeping 5 second (s)
Done sleeping...
total time taken to run 2 thrteads : 5.011707799974829
```

TASK 4 CUNCURRENT FUTURES

THREADPOOL EXECUTION BETTER AT TIME COMPLEXITY THAN THREADING

In [5]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something():
    print('sleeping  second ')
    time.sleep(10)
    return 'Done sleeping.'

with concurrent.futures.ThreadPoolExecutor() as executor:
    f1 = executor.submit(do_something)
    f2 = executor.submit(do_something)
    print(f1.result())
    print(f2.result())

finish = time.perf_counter()

print("total time taken to run 2 thrteads : {} ".format(start-finish))
```

```
sleeping  second
sleeping  second
Done sleeping.
Done sleeping.
total time taken to run 2 thrteads : -10.023355400015134
```

TASK 5 ||| CREATING MULTIPLE THREADS MANUMALLY WITHOUT FOR LOOP WITH DIFFERENT ARGS

In [6]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    return 'Done sleeping.'
```

```
with concurrent.futures.ThreadPoolExecutor() as executor:
    f1 = executor.submit(do_something, 1.5)
    f2 = executor.submit(do_something, 4.5)
    f3 = executor.submit(do_something, 2.5)
    f4 = executor.submit(do_something, 5.5)
    print(f1.result())
    print(f2.result())
    print(f3.result())
    print(f4.result())


finish = time.perf_counter()

print("total time taken to run 2 thrteads : {} ".format(finish-start))
```

```
sleeping 1.5 second (s)
sleeping 4.5 second (s)
sleeping 2.5 second (s)
sleeping 5.5 second (s)
Done sleeping.
Done sleeping.
Done sleeping.
Done sleeping.
total time taken to run 2 thrteads : 5.514934799983166
```

CREATING MULTIPLE THREADS WITH FOR LOOP WITH DIFFERENT ARGS

In [7]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something(seconds):
    print(f'sleeping {seconds} second (s)')
    time.sleep(seconds)
    return f'Done sleeping {seconds}...'

with concurrent.futures.ThreadPoolExecutor() as executor:
    secs = [5,4,3,2,1]
```

```
    results = [executor.submit(do_something, sec) for sec in secs]

    for f in concurrent.futures.as_completed(results):
        print(f.result())

finish = time.perf_counter()

print(f'Finished in {round(finish-start,2)} second(s)')
```

```
sleeping 5 second (s)sleeping 4 second (s)

sleeping 3 second (s)
sleeping 2 second (s)
sleeping 1 second (s)
Done sleeping 1...
Done sleeping 2...
Done sleeping 3...
Done sleeping 4...
Done sleeping 5...
Finished in 5.01 second(s)
```

USING MAP FUNCTION

In [8]:
```
import concurrent.futures
import time

start = time.perf_counter()


def do_something(seconds):
    print(f'Sleeping {seconds} second(s)...')
    time.sleep(seconds)
    return f'Done Sleeping...{seconds}'

with concurrent.futures.ThreadPoolExecutor() as executor:
    secs = [5, 4, 3, 2, 1]
    results = executor.map(do_something, secs)

for result in results:
    print(result)
```

```
finish = time.perf_counter()

print(f'Finished in {round(finish-start, 2)} second(s)')
```

```
Sleeping 5 second(s)...
Sleeping 4 second(s)...
Sleeping 3 second(s)...
Sleeping 2 second(s)...
Sleeping 1 second(s)...
Done Sleeping...5
Done Sleeping...4
Done Sleeping...3
Done Sleeping...2
Done Sleeping...1
Finished in 5.01 second(s)
```

TASK 6 ||| creating the multiple thread with the help of for loop

In [9]:
```
import concurrent.futures
import time

start = time.perf_counter()

def do_something():
    print('sleeping  second ')
    time.sleep(10)
    return 'Done sleeping.'

res=[executor.submit(do_something) for _ in range(2)]
for f in concurrent.futures.as_completed(res):
        print(f.result())


finish = time.perf_counter()

print("total time taken to run 2 thrteads : {} ".format(start-finish))
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[9], line 11
      8     time.sleep(10)
      9     return 'Done sleeping.'
---> 11 res=[executor.submit(do_something) for _ in range(2)]
     12 for f in concurrent.futures.as_completed(res):
     13         print(f.result())

File ~\anaconda3\Lib\concurrent\futures\thread.py:171, in ThreadPoolExecutor.submit(self, fn, *args, **kwargs)
    168     raise BrokenThreadPool(self._broken)
    170 if self._shutdown:
--> 171     raise RuntimeError('cannot schedule new futures after shutdown')
    172 if _shutdown:
    173     raise RuntimeError('cannot schedule new futures after '
    174                        'interpreter shutdown')

RuntimeError: cannot schedule new futures after shutdown
```

In [10]:

```python
import concurrent.futures
import time

start = time.perf_counter()

def do_something():
    print('sleeping second')
    time.sleep(10)
    return 'Done sleeping.'

#you must create and open the executor using a context manager
with concurrent.futures.ThreadPoolExecutor() as executor:
    res = [executor.submit(do_something) for _ in range(2)]

    for f in concurrent.futures.as_completed(res):
        print(f.result())

finish = time.perf_counter()


print("Total time taken to run 2 threads: {} seconds".format(finish - start))
```

```
sleeping second
sleeping second
Done sleeping.
Done sleeping.
Total time taken to run 2 threads: 10.093020100030117 seconds
```

TASK 7 ||| DOWNLOAD FILES

In [ ]:

In [11]:
```python
import time
import requests

img_urls = [
    'https://soundhelix.com',
    'https://soundhelix.com',
    'https://soundhelix.com',
    'https://soundhelix.com'
]

t1 = time.perf_counter()

for img_url in img_urls:
    img_bytes = requests.get(img_url).content
    img_name = img_url.split('/')[-1]
    img_name = f'{img_name}.mp3'
    with open(img_name, 'wb') as img_file:
        img_file.write(img_bytes)
        print(f'{img_name} was downloaded...')

t2 = time.perf_counter()
print(f'Finished in {t2-t1} seconds')
```

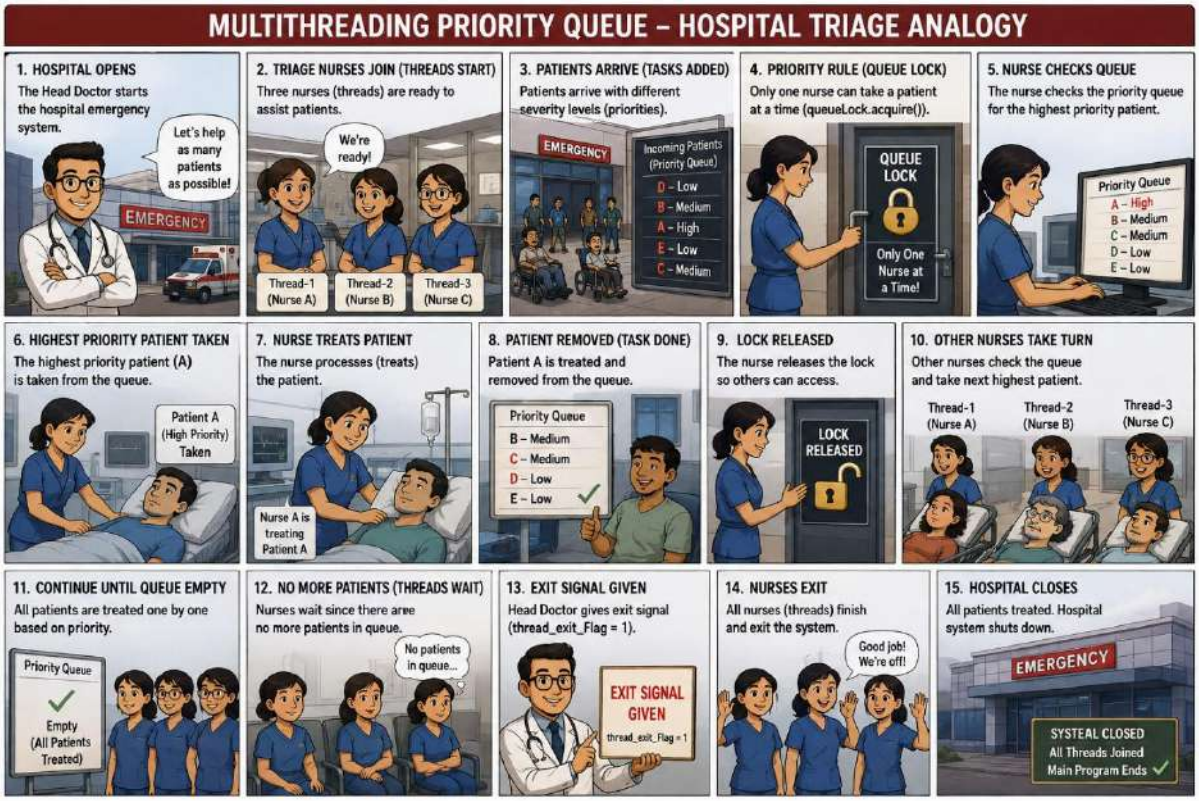
```
soundhelix.com.mp3 was downloaded...
soundhelix.com.mp3 was downloaded...
soundhelix.com.mp3 was downloaded...
soundhelix.com.mp3 was downloaded...
Finished in 33.106978600029834 seconds
```

In [ ]:

In [ ]:

TASK-3

Multithreading Priority Queue - Representative Diagram (conventional or non-conventional)



This diagram explains **multithreading using a simple hospital example**. The nurses represent threads, and the patients represent tasks waiting in the queue. The nurses take patients one by one and treat them. The **queue lock** makes sure that only one nurse takes a patient at a time, so there is no confusion. Once all patients are treated, the nurses finish their work and the program ends.

VIKNESH.G ||| 23MID0161

TASK 4

Practice printing MainThread name and worker threads

In [1]:
```
import threading
import time

names = ["Alice", "Bob", "Charlie", "David", "Eve"]

def worker(name):
    print("Worker Thread: {}".format(name))
    time.sleep(1)
    print("Task completed by {}".format(name))

print("Main Thread: {}".format(threading.current_thread().name))

threads = []

for i in range(5):
    t = threading.Thread(
        target=worker,
        args=(names[i],),
        name="Worker-{}".format(i + 1)
    )
    threads.append(t)
    t.start()

# ERROR: joining a thread that was never created
wrong_thread = threading.Thread(target=worker, args=("Wrong",))
threads[5].join()

print("All worker threads completed.")
```

```
Main Thread: MainThread
Worker Thread: Alice
Worker Thread: Bob
Worker Thread: Charlie
Worker Thread: David
Worker Thread: Eve
Task completed by Alice
Task completed by Charlie
Task completed by David
Task completed by Eve
Task completed by Bob
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[1], line 26
     24 # ERROR: joining a thread that was never created
     25 wrong_thread = threading.Thread(target=worker, args=("Wrong",))
---> 26 threads[5].join()
     28 print("All worker threads completed.")

IndexError: list index out of range
```

In [3]:

```python
import threading
import time

names = ["Alice", "Bob", "Charlie", "David", "Eve"]

def worker(name):
    print("Worker Thread: {}".format(threading.current_thread().name))
    print("Working on task for {}".format(name))
    time.sleep(1)
    print("Task completed by {}".format(name))

print("Main Thread: {}".format(threading.current_thread().name))

threads = []

for i in range(5):
    t = threading.Thread(
        target=worker,
        args=(names[i],),
        name="Worker-{}".format(i + 1)
    )
```

```
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("All worker threads completed.")
print("Main Thread: {}".format(threading.current_thread().name))
```

```
Main Thread: MainThread
Worker Thread: Worker-1
Working on task for Alice
Worker Thread: Worker-2
Working on task for Bob
Worker Thread: Worker-3
Working on task for Charlie
Worker Thread: Worker-4
Working on task for David
Worker Thread: Worker-5
Working on task for Eve
Task completed by AliceTask completed by Bob
Task completed by Charlie
Task completed by David
Task completed by Eve

All worker threads completed.
Main Thread: MainThread
```

The main thread creates and starts five worker threads. Each worker handles one name. join() makes the main thread wait until all workers finish.

In [ ]: